

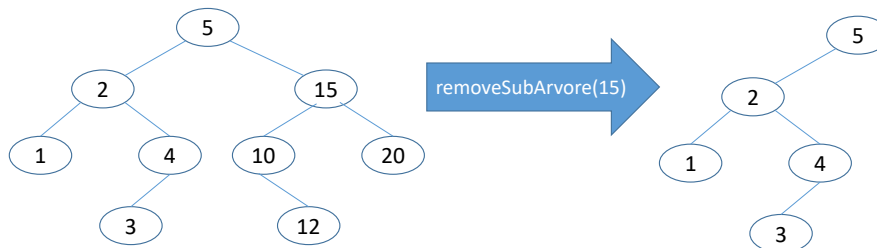
1–[55%] Crie uma classe genérica `Cofre` com as seguintes características:

- Podem ser acrescentados a um `Cofre` quaisquer valores comparáveis entre si, através de um método `add`. Estes valores podem ser removidos com um método `remove`, que recebe o valor a remover por parâmetro.
- O método `remove` deve ter complexidade linear. O método `add` deve ter complexidade constante.
- A classe `Cofre` deve ser iterável. O iterador percorre apenas o menor valor e o maior valor inseridos (só esses dois valores, por essa ordem). A remoção de valores através do iterador não é suportada. Deve gerar a excepção `ConcurrentModificationException` de forma adequada sempre que necessário.
- Deve ser disponibilizado um método estático `descobreCofre(obj)` que recebe um objeto `obj` por parâmetro e indica em que cofre é que esse valor está (ou `null` se não está em nenhum cofre). O método deve funcionar com complexidade constante. (pode assumir que um objeto nunca está em mais do que um `Cofre` simultaneamente).

2- [10%] Assuma que dispõe de um método `int pesquisa (int m[], int valor)` que efectua uma pesquisa binária. Este método devolve a posição em que o valor procurado se encontra, ou então um valor negativo ($-X$) caso este não esteja no array indicado. O valor de $\text{abs}(X+1)$ indica uma posição em que o valor procurado poderia ser inserido para preservar a ordem. **Este método já existe e não precisa de o fazer.** Construa um método que recebe um array de inteiros ordenado sem valores duplicados e um inteiro N . Este método verifica se N está mais próximo do seu antecessor (maior valor que é menor do que N) ou sucessor (menor valor que é maior do que N) no array, e devolve o valor correspondente (o antecessor ou sucessor, conforme o caso). Em caso de empate deve devolver o antecessor. O método deve ter complexidade logarítmica.

3–[35%] Considere uma classe `Arvore` que implementa uma árvore binária (construindo método recursivo e o respetivo método público de acesso):

- Crie um método recursivo `mostraMaiores` que recebe um valor e imprime todos os valores da árvore maiores do que esse valor. O método não deve realizar travessias da árvore desnecessárias.
- Crie um método recursivo `removeSubArvore` que recebe um valor e remove toda a sub-árvore com raiz nesse valor.



- Crie um método recursivo que devolve uma lista com todas as folhas da árvore. Não deve acrescentar outros parâmetros ao método, para além de um nodo da árvore.